ANÁLISE E PROJETO DE SISTEMAS

Tradutor Neox

Documentação de Análise e Projeto de Sistemas

Vitor Manoel

Novembro de 2026

Sumário

1. Visão Geral do Sistema — Tradutor Neox
    1. 1.1 Problema
    2. 1.2 Proposta
    3. 1.3 Análise × Projeto
    4. 1.4 Etapas do desenvolvimento aplicadas ao projeto
    5. 1.5 Tecnologias
2. Engenharia de Requisitos
    1. 2.1 Briefing do cliente
    2. 2.2 Stakeholders
    3. 2.3 Elicitação
    4. 2.4 Requisitos Funcionais (RF)
    5. 2.5 Requisitos Não Funcionais (RNF)
    6. 2.6 Regras de Negócio (RN)
    7. 2.7 Rastreabilidade: requisitos × implementação
3. Casos de Uso
    1. 3.1 Atores
    2. 3.2 Lista de casos de uso
    3. 3.3 Diagrama de casos de uso
    4. 3.4 Documentação dos casos de uso
4. Classes de Análise
    1. 4.1 Substantivos candidatos
    2. 4.2 Diagrama de classes de análise
    3. 4.3 Leitura das associações
5. Diagrama de Classes de Projeto
    1. 5.1 Da análise ao projeto
    2. 5.2 Diagrama
    3. 5.3 Classes
    4. 5.4 Relacionamentos (aplicando a árvore de decisão)
    5. 5.5 Checklist de refinamento
    6. 5.6 Correspondência com o código
6. Versionamento com Git
    1. 6.1 Configuração
    2. 6.2 Commits semânticos (Conventional Commits)
    3. 6.3 Branches

1. Visão Geral do Sistema — Tradutor Neox

1.1 Problema

Pessoas que precisam se comunicar em outro idioma (estudantes, viajantes, criadores de conteúdo) muitas vezes não só querem ler a tradução, mas também **ouvir** como ela soa, com uma voz natural e com uma entonação adequada ao contexto (alegre, triste, séria etc.). Ferramentas comuns traduzem o texto, mas oferecem pouca ou nenhuma personalização da voz.

1.2 Proposta

O **Tradutor Neox** é uma aplicação web que:

1. recebe um texto digitado pelo usuário;
2. traduz o texto para o idioma escolhido, usando inteligência artificial;
3. converte a tradução em áudio (síntese de voz — *text-to-speech*);
4. permite escolher a **voz**, a **emoção** e uma **personalidade** para a fala.

1.3 Análise × Projeto

	Pergunta	No Tradutor Neox
Análise	*O quê?*	O usuário precisa digitar um texto, escolher o idioma de destino e ouvir a tradução em voz alta, podendo escolher voz e emoção.
Projeto	*Como?*	Aplicação Next.js (React + TypeScript). O front-end chama rotas de API internas (`/api/translate`, `/api/tts`, `/api/voices`), que se comunicam com a API da OpenAI (modelo `gpt-4o-mini` para tradução e `tts-1` para voz). A emoção é aplicada no navegador via Web Audio API (velocidade, tom e volume).

1.4 Etapas do desenvolvimento aplicadas ao projeto

Etapa	O que acontece no Tradutor Neox
Especificação	Definiu-se que o sistema deve traduzir textos para vários idiomas e reproduzir o resultado em áudio, com escolha de voz e emoção. Restrições: uso da API da OpenAI (custo por requisição), chave de API protegida no servidor, interface web acessível pelo navegador.
Desenvolvimento	Front-end em React (`app/page.tsx`), rotas de API no servidor Next.js (`app/api/*`), integração com OpenAI e manipulação de áudio com Web Audio API. Código versionado com Git e Conventional Commits.
Validação	Validação de entrada nas APIs (texto vazio ou idioma ausente retornam erro 400), verificação da chave (erro 500 "OPENAI_API_KEY ausente"), rota de diagnóstico `/api/diag` que testa se a chave é válida, e testes manuais de tradução e reprodução de áudio.

Etapa	O que acontece no Tradutor Neox
Evolução	Funcionalidades planejadas/adicionadas depois da primeira versão: emoções na fala, personalidade, prévia de vozes, presets salvos no navegador, controle de velocidade e limite de requisições (*rate limit*) por IP.

1.5 Tecnologias

Camada	Tecnologia
Interface	React 18, Next.js 14 (App Router), TypeScript
Servidor	Rotas de API do Next.js (Node.js ≥ 18.18)
Inteligência artificial	OpenAI — `gpt-4o-mini` (tradução), `tts-1` (voz)
Áudio	Web Audio API (navegador)
Versionamento	Git + GitHub, Conventional Commits
Modelagem	Draw.io (diagramas UML)

2. Engenharia de Requisitos

2.1 Briefing do cliente

> Estudantes, viajantes e criadores de conteúdo precisam traduzir textos curtos e, além de ler a tradução, **ouvir** como ela soa. As ferramentas atuais traduzem, mas oferecem pouca personalização da voz. O cliente quer uma aplicação web, sem cadastro, em que o usuário digite um texto, escolha o idioma de destino e ouça a tradução com a voz, a emoção e a velocidade que preferir. O usuário também quer guardar as combinações que mais usa (favoritos) para não precisar configurá-las de novo. A tradução e a voz são geradas por um serviço externo de inteligência artificial (OpenAI), cujo custo é por requisição, por isso o uso precisa ser limitado e a chave de acesso ao serviço deve ficar protegida.

2.2 Stakeholders

Stakeholder	Interesse
Usuário	Traduzir e ouvir textos de forma rápida e personalizada
Administrador do sistema	Manter a aplicação no ar, configurar a chave da API e controlar custos
Serviço OpenAI (externo)	Fornecer a tradução (`gpt-4o-mini`) e a síntese de voz (`tts-1`)

2.3 Elicitação

Brainstorming (ideias levantadas e decisão de escopo)

Ideia	Decisão
Traduzir texto para vários idiomas	✅ No escopo (RF-01)
Ouvir o texto em voz alta	✅ No escopo (RF-02)
Escolher entre várias vozes e ouvir uma prévia	✅ No escopo (RF-03, RF-04)
Dar emoção à fala (feliz, triste...)	✅ No escopo (RF-05)
Adaptar o tom da tradução a uma personalidade	✅ No escopo (RF-06)
Controlar a velocidade da fala	✅ No escopo (RF-07)
Salvar configurações favoritas	✅ No escopo (RF-08 a RF-10)
Cadastro de usuários e login	❌ Fora do escopo — aumenta o atrito e exige armazenar dados pessoais
Histórico de traduções no servidor	❌ Fora do escopo — conflita com a privacidade (RNF-08)

Ideia	Decisão
Reconhecimento de voz (falar em vez de digitar)	⏳ Evolução futura

Entrevista (resumo)

Pergunta	Resposta do cliente
Que tamanho de texto será traduzido? (fechada)	Frases e parágrafos curtos, até cerca de uma página (≈1000 caracteres).
Quantos idiomas são necessários? (fechada)	Pelo menos os 20 mais falados; hoje são 25.
O que mais incomoda nas ferramentas atuais? (aberta)	A voz robótica e sem emoção, e ter que configurar tudo de novo a cada uso.
Como evitar custos inesperados com a API? (aberta)	Limitar quantas requisições cada visitante pode fazer em um intervalo de tempo.

Cenário

> *Ana vai apresentar um trabalho em espanhol. Ela abre o Tradutor Neox, digita o parágrafo de abertura em português e escolhe "Espanhol" como idioma de tradução. Na personalidade, escreve "professora paciente". Ela clica em "Escolher voz", ouve a prévia de duas vozes e escolhe a "Nova". Seleciona a emoção "Sereno", reduz a velocidade para 0,85x e clica em "Traduzir & Falar". O sistema mostra o texto traduzido e reproduz o áudio. Como vai treinar todos os dias, Ana salva a configuração como o favorito "Apresentação espanhol". No dia seguinte, ela aplica o favorito e só precisa digitar o novo texto.*

Esse cenário revela os requisitos: traduzir, escolher idioma, personalidade, escolher voz, ouvir prévia, aplicar emoção, ajustar velocidade, exibir tradução, falar e salvar/aplicar favorito.

Requisito de usuário × requisito de sistema

Visão	Exemplo
Requisito de usuário	*"O sistema deve permitir que o usuário ouça a tradução do seu texto com a voz que escolher."*
Requisito de sistema	*"Ao acionar 'Traduzir & Falar', o sistema deve enviar o texto (até 1000 caracteres) e o idioma de destino para* `/api/translate`*, receber o texto traduzido, enviá-lo com o identificador da voz para* `/api/tts` *e reproduzir o áudio MP3 retornado em até 8 segundos."*

2.4 Requisitos Funcionais (RF)

RF-01
Nome: Traduzir texto

Descrição: O sistema deve traduzir o texto digitado pelo usuário para o idioma de destino selecionado entre os idiomas suportados (RN-03) e exibir o texto traduzido na tela.

RF-02
Nome: Falar texto
Descrição: O sistema deve converter em áudio (síntese de voz) o texto traduzido — ou o texto original, quando a opção "Sem tradução" estiver selecionada — e reproduzi-lo automaticamente no navegador.

RF-03
Nome: Escolher voz
Descrição: O sistema deve listar as vozes disponíveis (RN-04), com nome e descrição, e permitir que o usuário selecione a voz usada na fala. A voz padrão é "Alloy".

RF-04
Nome: Ouvir prévia da voz
Descrição: Na janela de escolha de voz, o sistema deve reproduzir, a pedido do usuário, a frase "Esta é uma prévia da voz [nome]" com a voz correspondente.

RF-05
Nome: Aplicar emoção à fala
Descrição: O sistema deve permitir que o usuário selecione uma emoção (RN-05). Quando a emoção for diferente de "Neutro", o sistema deve alterar velocidade, tom e volume do áudio no navegador conforme a emoção escolhida.

RF-06
Nome: Definir personalidade da tradução
Descrição: O sistema deve permitir que o usuário informe uma personalidade (ex.: "formal", "professor paciente") que será usada para ajustar o tom e o estilo da tradução, sem alterar o sentido do texto (RN-06).

RF-07
Nome: Ajustar velocidade de reprodução
Descrição: O sistema deve permitir que o usuário ajuste a velocidade de reprodução do áudio por meio de um controle deslizante, exibindo o valor atual (ex.: "1,00x"), dentro da faixa definida na RN-07.

RF-08
Nome: Salvar favorito
Descrição: O sistema deve permitir que o usuário salve a configuração atual (voz, emoção, personalidade, idioma e velocidade) como um favorito identificado por um nome (RN-08, RN-09).

RF-09
Nome: Aplicar favorito
Descrição: O sistema deve listar os favoritos salvos e, quando o usuário escolher um deles, preencher automaticamente voz, emoção, personalidade, idioma e velocidade com os valores do favorito.

RF-10
Nome: Excluir favorito
Descrição: O sistema deve permitir que o usuário exclua um favorito da lista.

RF-11
Nome: Contar caracteres
Descrição: O sistema deve exibir, abaixo do campo de texto, a quantidade de caracteres digitados e o limite (ex.: "14/1000 caracteres"), destacando o contador em vermelho e desabilitando o botão de envio quando o limite for ultrapassado.

RF-12
Nome: Exibir mensagens de erro
Descrição: O sistema deve exibir na tela uma mensagem que descreva o problema sempre que uma tradução, síntese de voz ou prévia falhar (texto vazio, texto longo, limite de requisições, falha do serviço externo).

RF-13
Nome: Verificar diagnóstico do sistema
Descrição: O sistema deve disponibilizar a rota `/api/diag`, que informa se a chave da API está configurada (`hasKey`), se ela é aceita pelo serviço externo (`modelsOk`) e o erro retornado, quando houver.

2.5 Requisitos Não Funcionais (RNF)

Código	Tipo
RNF-01 a RNF-07	De produto
RNF-09 a RNF-10	Organizacionais
RNF-08, RNF-11	Externos

RNF-01 — *Produto (desempenho)*
Nome: Tempo de resposta da tradução
Descrição: Para textos de até 1000 caracteres, a tradução deve ser exibida em até 5 segundos em 95% das requisições, medidos do clique no botão até a exibição do texto traduzido.

RNF-02 — *Produto (desempenho)*
Nome: Tempo de geração do áudio
Descrição: Para textos de até 1000 caracteres, o áudio deve começar a tocar em até 8 segundos após o clique em "Falar" em 95% das requisições.

RNF-03 — *Produto (usabilidade)*
Nome: Facilidade de uso
Descrição: Um usuário que nunca usou o sistema deve conseguir traduzir e ouvir um texto em até 60 segundos, sem treinamento, usando no máximo 3 ações (digitar o texto, escolher o idioma, clicar em "Traduzir & Falar").

RNF-04 — *Produto (portabilidade)*
Nome: Layout responsivo
Descrição: A interface deve funcionar em telas a partir de 360 px de largura sem rolagem horizontal, e nas duas versões mais recentes do Chrome, Edge, Firefox e Safari.

RNF-05 — *Produto (acessibilidade)*
Nome: Acessibilidade

Descrição: Todos os campos devem ter rótulo associado; todas as funções devem ser acessíveis pelo teclado; as janelas de vozes e de favoritos devem fechar com a tecla ESC, devolvendo o foco ao botão que as abriu.

RNF-06 — *Produto (confiabilidade)*
Nome: Tolerância a falhas do serviço externo
Descrição: Uma falha do serviço de tradução ou de voz não pode travar a interface: em 100% dos casos o botão deve voltar a ficar disponível e uma mensagem de erro deve ser exibida (RF-12).

RNF-07 — *Produto (segurança)*
Nome: Proteção da chave da API
Descrição: A chave da API da OpenAI deve ser lida apenas no servidor, a partir da variável de ambiente `OPENAI_API_KEY`, e nunca pode ser enviada ao navegador nem versionada no Git (o arquivo `.env.local` deve constar no `.gitignore`).

RNF-08 — *Externo (legal — LGPD)*
Nome: Privacidade dos dados
Descrição: O sistema não deve exigir cadastro nem armazenar no servidor os textos digitados, as traduções ou os áudios gerados. Os favoritos devem ficar salvos somente no navegador do usuário (`localStorage`).

RNF-09 — *Organizacional (tecnologia)*
Nome: Plataforma de desenvolvimento
Descrição: O sistema deve ser desenvolvido em Next.js 14 com React 18 e TypeScript, executando em Node.js 18.18 ou superior, e publicado na Vercel.

RNF-10 — *Organizacional (processo)*
Nome: Versionamento e modelagem
Descrição: O código deve ser versionado no Git/GitHub seguindo o padrão Conventional Commits, com uma branch por tarefa (`feat/`, `fix/`, `docs/`...). Os diagramas UML devem ser produzidos no Draw.io.

RNF-11 — *Externo (ético)*
Nome: Transparência sobre voz sintética
Descrição: A interface deve informar ao usuário que a voz reproduzida é gerada por inteligência artificial, conforme as políticas de uso do serviço de voz da OpenAI.

2.6 Regras de Negócio (RN)

RN-01
Nome: Tamanho máximo do texto
Descrição: O texto original deve ter no máximo 1000 caracteres. O texto enviado para síntese de voz deve ter no máximo 4096 caracteres (limite do serviço de voz).

RN-02
Nome: Limite de requisições
Descrição: Cada endereço IP pode fazer no máximo 60 requisições de tradução e de voz (somadas) a cada 10 minutos. Ao exceder o limite, o sistema deve recusar a requisição (HTTP 429) e informar em quantos segundos o usuário poderá tentar novamente.

RN-03
Nome: Idiomas suportados
Descrição: A tradução só pode ser feita para um dos 25 idiomas cadastrados no sistema. Quando a opção "Sem tradução" estiver selecionada, o texto original é falado sem passar pela tradução.

RN-04
Nome: Vozes permitidas
Descrição: A síntese só pode usar uma das 6 vozes cadastradas: Alloy, Echo, Fable, Onyx, Nova e Shimmer. Qualquer outro valor deve ser recusado.

RN-05
Nome: Emoções disponíveis
Descrição: As emoções permitidas são Neutro, Feliz, Triste, Sereno e Energético. A emoção padrão é Neutro, que reproduz o áudio sem alterações.

RN-06
Nome: Personalidade
Descrição: A personalidade deve ter no máximo 100 caracteres e só é aplicada quando houver um idioma de tradução selecionado.

RN-07
Nome: Faixa de velocidade
Descrição: A velocidade de reprodução deve estar entre 0,60x e 1,60x, em passos de 0,05x. O valor padrão é 1,00x.

RN-08
Nome: Quantidade máxima de favoritos
Descrição: O usuário pode ter no máximo 40 favoritos. O favorito mais recente fica no topo da lista; ao salvar o 41º, o mais antigo é descartado.

RN-09
Nome: Nome do favorito
Descrição: Todo favorito deve ter um nome não vazio, e não podem existir dois favoritos com o mesmo nome (sem diferenciar maiúsculas de minúsculas).

RN-10
Nome: Texto obrigatório
Descrição: O sistema não deve traduzir nem falar um texto vazio ou composto apenas por espaços.

2.7 Rastreabilidade: requisitos × implementação

Requisito	Onde está implementado
RF-01, RF-06, RN-03, RN-06	`app/api/translate/route.ts`
RF-02, RN-04	`app/api/tts/route.ts`
RF-03	`app/api/voices/route.ts`, `lib/constants.ts`
RF-05, RF-07	`lib/audio.ts`, `app/page.tsx`

Requisito	Onde está implementado
RF-08 a RF-10, RN-08, RN-09	`lib/presets.ts`, `app/page.tsx`
RF-11, RF-12, RN-10	`app/page.tsx`
RF-13	`app/api/diag/route.ts`
RN-01, RN-05, RN-07	`lib/constants.ts`
RN-02	`lib/ratelimit.ts`
RNF-07	`.gitignore`, variáveis de ambiente

3. Casos de Uso

3.1 Atores

Ator	Tipo	Papel
Usuário	Principal	Digita o texto, escolhe idioma, voz, emoção, personalidade e velocidade, ouve a fala e gerencia favoritos
Administrador	Principal (generalização de Usuário)	Tudo que o Usuário faz e, além disso, verifica o diagnóstico do sistema (chave da API)
OpenAI `«sistema»`	Secundário (sistema externo)	Executa a tradução (`gpt-4o-mini`) e a síntese de voz (`tts-1`), e valida a chave da API

> **Generalização:** o `Administrador` é um `Usuário` — herda todas as associações dele e acrescenta o caso de uso `Verificar Diagnóstico`. Por isso a seta de generalização aponta do `Administrador` para o `Usuário`.

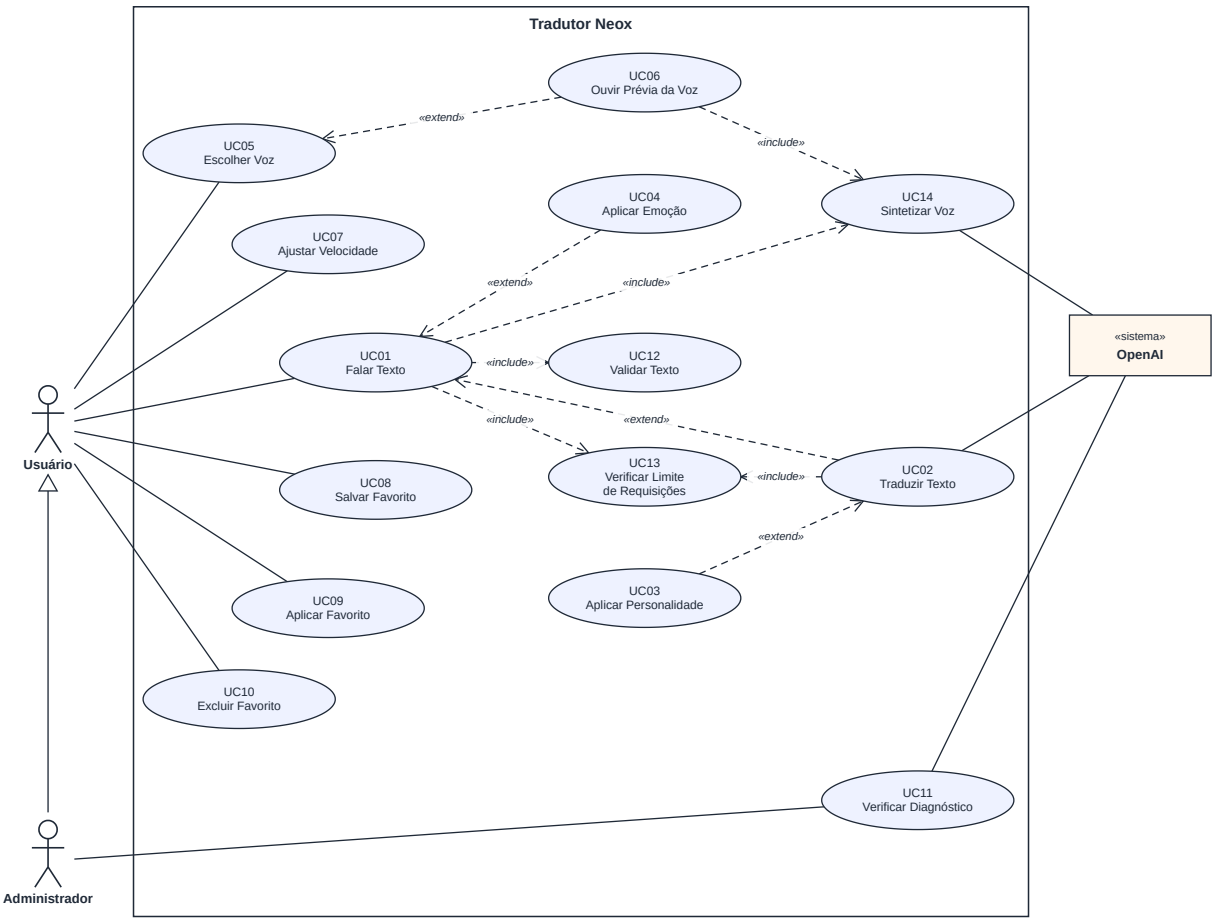
3.2 Lista de casos de uso

Código	Caso de uso	Tipo	Requisitos
UC01	Falar Texto	Primário	RF-02, RF-11, RF-12, RN-10
UC02	Traduzir Texto	Primário (extensão de UC01)	RF-01, RN-03
UC03	Aplicar Personalidade	Extensão de UC02	RF-06, RN-06
UC04	Aplicar Emoção	Extensão de UC01	RF-05, RN-05
UC05	Escolher Voz	Primário	RF-03, RN-04
UC06	Ouvir Prévia da Voz	Extensão de UC05	RF-04
UC07	Ajustar Velocidade	Primário	RF-07, RN-07
UC08	Salvar Favorito	Primário	RF-08, RN-08, RN-09
UC09	Aplicar Favorito	Primário	RF-09
UC10	Excluir Favorito	Secundário	RF-10
UC11	Verificar Diagnóstico	Secundário	RF-13
UC12	Validar Texto	Incluído por UC01	RN-01, RN-10
UC13	Verificar Limite de Requisições	Incluído por UC01 e UC02	RN-02
UC14	Sintetizar Voz	Incluído por UC01 e UC06	RF-02, RN-04

Relacionamentos

Relacionamento	Tipo	Justificativa
UC01 → UC12	`«include»`	Todo texto é validado antes de ser falado
UC01 → UC13 / UC02 → UC13	`«include»`	Toda requisição ao serviço externo passa pelo limite por IP
UC01 → UC14 / UC06 → UC14	`«include»`	Falar o texto e ouvir a prévia sempre geram áudio pelo mesmo serviço de síntese
UC02 → UC01	`«extend»`	**Condição:** um idioma de destino está selecionado. Com "Sem tradução", o texto original é falado
UC03 → UC02	`«extend»`	**Condição:** o usuário informou uma personalidade
UC04 → UC01	`«extend»`	**Condição:** a emoção escolhida é diferente de "Neutro"
UC06 → UC05	`«extend»`	**Condição:** o usuário pede para ouvir a prévia antes de escolher

3.3 Diagrama de casos de uso



Arquivo editável no Draw.io: `diagramas/casos-de-uso.drawio`

Como ler: o `Usuário` se associa aos objetivos que ele inicia (falar, escolher voz, ajustar velocidade e gerenciar favoritos). `Traduzir Texto`, `Aplicar Emoção` e `Aplicar Personalidade` só acontecem em certas condições, por isso são `«extend»`. `Validar Texto`, `Verificar Limite de Requisições` e `Sintetizar Voz` acontecem sempre que o caso de uso base é executado, por isso são `«include»`. A `OpenAI` fica fora da fronteira do sistema: é um agente externo que participa da tradução, da síntese e do diagnóstico.

3.4 Documentação dos casos de uso

UC01 — Falar Texto

Campo	Conteúdo
Nome	Falar Texto
Ator(es)	Usuário (principal), OpenAI (secundário, via UC14)
Descrição	O usuário digita um texto e o sistema o reproduz em voz alta com a voz, a emoção e a velocidade configuradas, traduzindo-o antes quando houver idioma selecionado.
Pré-condições	A lista de vozes foi carregada.
Fluxo principal	1. Usuário digita o texto. 2. Sistema exibe o contador de caracteres (RF-11). 3. Usuário clica em "Falar" (ou "Traduzir & Falar"). 4. Sistema executa `Validar Texto` (UC12). 5. Sistema executa `Verificar Limite de Requisições` (UC13). 6. Sistema executa `Sintetizar Voz` (UC14) com o texto e a voz escolhida. 7. Sistema reproduz o áudio na velocidade configurada.
Fluxos alternativos	3a. Se houver idioma de destino selecionado, o sistema executa `Traduzir Texto` (UC02) antes do passo 6 e fala o texto traduzido. 7a. Se a emoção for diferente de "Neutro", o sistema executa `Aplicar Emoção` (UC04) durante a reprodução.
Exceções	4a. Texto vazio ou com mais de 1000 caracteres: o sistema exibe a mensagem de erro e não envia a requisição. 5a. Limite de requisições excedido: o sistema exibe "Limite de requisições atingido. Tente novamente em N segundos." 6a. Falha no serviço de voz: o sistema exibe "Falha ao gerar áudio" e libera o botão.
Pós-condições	O áudio foi reproduzido e fica disponível no player para ser ouvido novamente.

UC02 — Traduzir Texto

Campo	Conteúdo
Nome	Traduzir Texto
Ator(es)	Usuário (indireto, por UC01), OpenAI (secundário)
Descrição	O sistema traduz o texto do usuário para o idioma de destino escolhido.

Campo	Conteúdo
Pré-condições	Um idioma de destino está selecionado e o texto já foi validado (UC12).
Fluxo principal	1. Sistema executa `Verificar Limite de Requisições` (UC13). 2. Sistema confirma que o idioma está entre os 25 suportados (RN-03). 3. Sistema envia o texto e o idioma ao serviço de tradução da OpenAI. 4. OpenAI devolve o texto traduzido. 5. Sistema exibe o texto traduzido na seção "Tradução".
Fluxos alternativos	3a. Se houver personalidade informada, o sistema executa `Aplicar Personalidade` (UC03), incluindo a instrução de tom na tradução.
Exceções	2a. Idioma não suportado: o sistema retorna "Idioma de destino não suportado". 4a. Falha no serviço de tradução: o sistema exibe "Falha na tradução" e não segue para a síntese.
Pós-condições	O texto traduzido está disponível para ser falado (UC01).

UC05 — Escolher Voz

Campo	Conteúdo
Nome	Escolher Voz
Ator(es)	Usuário
Descrição	O usuário escolhe, entre as 6 vozes disponíveis, a voz usada na fala.
Pré-condições	A lista de vozes foi carregada.
Fluxo principal	1. Usuário clica em "Escolher voz". 2. Sistema abre a janela com as vozes, cada uma com nome e descrição. 3. Usuário clica em "Usar esta voz". 4. Sistema fecha a janela e exibe a voz escolhida ao lado do rótulo "Voz".
Fluxos alternativos	3a. Usuário clica em "Prévia": o sistema executa `Ouvir Prévia da Voz` (UC06) e volta ao passo 3. 3b. Usuário pressiona ESC ou clica em "Fechar": a janela fecha sem alterar a voz.
Exceções	2a. Falha ao carregar as vozes: o sistema exibe "Não foi possível carregar as vozes."
Pós-condições	A voz escolhida será usada nas próximas falas.

UC06 — Ouvir Prévia da Voz

Campo	Conteúdo
Nome	Ouvir Prévia da Voz
Ator(es)	Usuário, OpenAI (secundário, via UC14)
Descrição	O sistema reproduz uma frase curta com a voz selecionada para que o usuário a conheça antes de escolher.

Campo	Conteúdo
Pré-condições	A janela de escolha de voz está aberta.
Fluxo principal	1. Usuário clica em "Prévia" em uma voz. 2. Sistema executa `Sintetizar Voz` (UC14) com a frase "Esta é uma prévia da voz [nome]". 3. Sistema reproduz o áudio na velocidade configurada.
Fluxos alternativos	—
Exceções	2a. Falha no serviço de voz ou limite excedido: o sistema exibe a mensagem de erro correspondente.
Pós-condições	Nenhuma alteração de configuração.

UC08 — Salvar Favorito

Campo	Conteúdo
Nome	Salvar Favorito
Ator(es)	Usuário
Descrição	O usuário salva a configuração atual de fala com um nome, para reutilizá-la depois.
Pré-condições	Nenhuma.
Fluxo principal	1. Usuário clica em "Salvar como favorito". 2. Sistema pede o nome do favorito. 3. Usuário informa o nome e confirma. 4. Sistema valida o nome (RN-09). 5. Sistema salva voz, emoção, personalidade, idioma e velocidade no topo da lista de favoritos, no navegador. 6. Sistema atualiza o contador "Favoritos (N)".
Fluxos alternativos	3a. Usuário cancela: nada é salvo. 5a. Já existem 40 favoritos: o mais antigo é descartado (RN-08).
Exceções	4a. Nome vazio: o sistema exibe "Informe um nome para o favorito." 4b. Nome repetido: o sistema exibe "Já existe um favorito com esse nome."
Pós-condições	O favorito aparece na lista e continua salvo ao recarregar a página.

UC09 — Aplicar Favorito

Campo	Conteúdo
Nome	Aplicar Favorito
Ator(es)	Usuário
Descrição	O usuário escolhe um favorito e o sistema preenche a configuração de fala com os valores salvos.

Campo	Conteúdo
Pré-condições	Existe pelo menos um favorito salvo.
Fluxo principal	1. Usuário clica em "Favoritos". 2. Sistema lista os favoritos com nome, voz, emoção, velocidade, idioma e personalidade. 3. Usuário clica em "Aplicar". 4. Sistema preenche voz, emoção, personalidade, idioma e velocidade e fecha a janela.
Fluxos alternativos	2a. Não há favoritos: o sistema exibe "Nenhum favorito salvo ainda." 3a. Usuário pressiona ESC: a janela fecha sem alterações.
Exceções	—
Pós-condições	A configuração de fala está igual à do favorito escolhido.

UC10 — Excluir Favorito

Campo	Conteúdo
Nome	Excluir Favorito
Ator(es)	Usuário
Descrição	O usuário remove um favorito da lista.
Pré-condições	Existe pelo menos um favorito salvo.
Fluxo principal	1. Usuário abre a janela de favoritos. 2. Usuário clica em "Excluir" no favorito desejado. 3. Sistema remove o favorito e atualiza a lista e o contador.
Fluxos alternativos	—
Exceções	—
Pós-condições	O favorito não aparece mais na lista, nem após recarregar a página.

UC11 — Verificar Diagnóstico

Campo	Conteúdo
Nome	Verificar Diagnóstico
Ator(es)	Administrador (principal), OpenAI (secundário)
Descrição	O administrador confere se a chave da API está configurada e se é aceita pela OpenAI.
Pré-condições	O sistema está publicado.
Fluxo principal	1. Administrador acessa `/api/diag`. 2. Sistema verifica se a variável `OPENAI_API_KEY` existe. 3. Sistema solicita à OpenAI a lista de modelos para validar a chave. 4. Sistema retorna `hasKey`, `modelsOk` e `error`.

Campo	Conteúdo
Fluxos alternativos	2a. Chave ausente: retorna `hasKey: false` sem consultar a OpenAI.
Exceções	3a. Chave recusada: retorna `modelsOk: false` com o nome, a mensagem e o status do erro.
Pós-condições	O administrador sabe se o sistema consegue usar o serviço externo.

Casos de uso incluídos e extensões (resumo)

Caso de uso	Fluxo resumido
UC03 — Aplicar Personalidade	Acrescenta à instrução de tradução: "Adapte o tom e o estilo da tradução a esta personalidade, sem alterar o sentido do texto". Recusa personalidade com mais de 100 caracteres.
UC04 — Aplicar Emoção	Na reprodução, altera velocidade, tom (detune), volume e realce de graves conforme a emoção: Feliz (1,10x, +1 semitom), Triste (0,90x, −1 semitom), Sereno (0,95x), Energético (1,20x, +½ semitom).
UC07 — Ajustar Velocidade	O usuário move o controle deslizante entre 0,60x e 1,60x; o sistema exibe o valor e aplica ao áudio atual e aos próximos.
UC12 — Validar Texto	Recusa texto vazio (RN-10) ou com mais de 1000 caracteres (RN-01), exibindo a mensagem de erro.
UC13 — Verificar Limite de Requisições	Conta as requisições do IP na janela de 10 minutos; acima de 60, responde HTTP 429 com o tempo de espera (RN-02).
UC14 — Sintetizar Voz	Valida a voz (RN-04) e o tamanho do texto (até 4096 caracteres), envia ao modelo `tts-1` e devolve o áudio MP3.

4. Classes de Análise

Visão **estrutural** do sistema: quais conceitos existem no domínio do Tradutor Neox e como se relacionam. Nesta etapa as classes têm apenas nome, atributos e métodos em alto nível, sem tipos nem visibilidade.

4.1 Substantivos candidatos

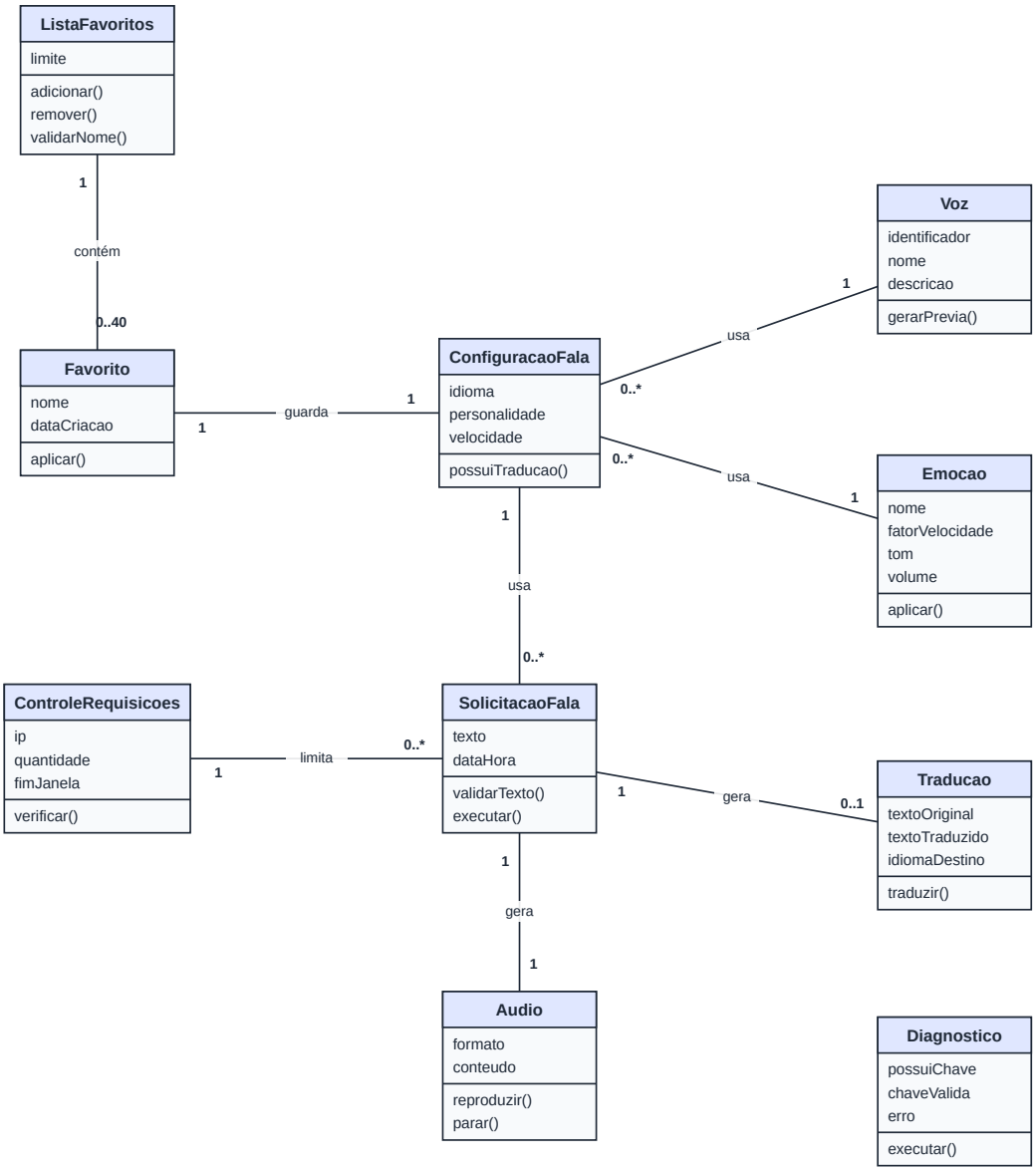
Substantivos grifados nos requisitos (seção 2) e nos casos de uso (seção 3), filtrados pelas heurísticas da aula:

Substantivo candidato	Classe ou atributo?	Justificativa
Usuário	Descartado	É ator, mas o sistema **não guarda dados** sobre ele (não há cadastro — RNF-08)
Administrador	Descartado	Mesmo motivo do Usuário
Texto	Atributo	Vira `texto` de `SolicitacaoFala`
Solicitação de fala	Classe	Representa a transação "falar este texto com esta configuração"; tem dados próprios (texto, data/hora) e comportamento (validar, executar)
Tradução	Classe	Tem dados próprios (texto original, texto traduzido, idioma) e é gerada a partir de uma solicitação
Idioma	Atributo	Na análise é apenas um nome (`idioma` de `ConfiguracaoFala`, `idiomaDestino` de `Traducao`)
Voz	Classe	Tem dados próprios (identificador, nome, descrição) e existe independente de qualquer configuração
Prévia	Método	Não guarda dados: é o comportamento `gerarPrevia()` de `Voz`
Emoção	Classe	Tem dados próprios (fator de velocidade, tom, volume) e comportamento (`aplicar()`)
Personalidade	Atributo	É só um texto livre da configuração
Velocidade	Atributo	É só um número da configuração
Configuração de fala	Classe	Agrupa voz, emoção, idioma, personalidade e velocidade escolhidos
Favorito	Classe	Guarda uma configuração com nome e data de criação
Lista de favoritos	Classe	Tem regra própria (limite de 40, nome único) e comportamento (adicionar, remover)
Áudio	Classe	Tem dados próprios (formato, conteúdo) e comportamento (reproduzir, parar)

Substantivo candidato	Classe ou atributo?	Justificativa
Limite de requisições	Classe (`ControleRequisicoes`)	Guarda contagem por IP e fim da janela, e tem comportamento (`verificar()`)
Mensagem de erro	Descartado	É saída da interface, não um conceito do domínio
Diagnóstico	Classe	Tem dados próprios (possui chave, chave válida, erro)
OpenAI	Descartado na análise	É ator externo; vira a classe de fronteira `ServicoIA` no projeto (seção 5)
Navegador	Descartado	Fora do escopo — é o ambiente de execução
Caractere	Descartado	É apenas a unidade de medida do contador

> **Sinônimos tratados:** "favorito" e "preset" são o mesmo conceito (usamos **Favorito**); "fala", "síntese" e "áudio gerado" foram separados em `SolicitacaoFala` (o pedido) e `Audio` (o resultado).

4.2 Diagrama de classes de análise



Arquivo editável no Draw.io: `diagramas/classes-analise.drawio`

4.3 Leitura das associações

Associação	Multiplicidade	Leitura
ListaFavoritos — Favorito	`1` — `0..40`	Uma lista contém de zero a 40 favoritos; cada favorito pertence a uma lista
Favorito — ConfiguracaoFala	`1` — `1`	Cada favorito guarda exatamente uma configuração
ConfiguracaoFala — Voz	`0..*` — `1`	Cada configuração usa uma voz; uma voz pode estar em várias configurações

Associação	Multiplicidade	Leitura
ConfiguracaoFala — Emocao	`0..*` — `1`	Cada configuração usa uma emoção; uma emoção pode estar em várias configurações
SolicitacaoFala — ConfiguracaoFala	`0..*` — `1`	Cada solicitação usa uma configuração
SolicitacaoFala — Traducao	`1` — `0..1`	Uma solicitação gera no máximo uma tradução (zero quando "Sem tradução")
SolicitacaoFala — Audio	`1` — `1`	Toda solicitação concluída gera um áudio
ControleRequisicoes — SolicitacaoFala	`1` — `0..*`	O controle de um IP limita as várias solicitações feitas por ele

> `Diagnostico` não se associa às demais classes do domínio: ele só se relaciona com o serviço externo, o que aparece no diagrama de projeto.

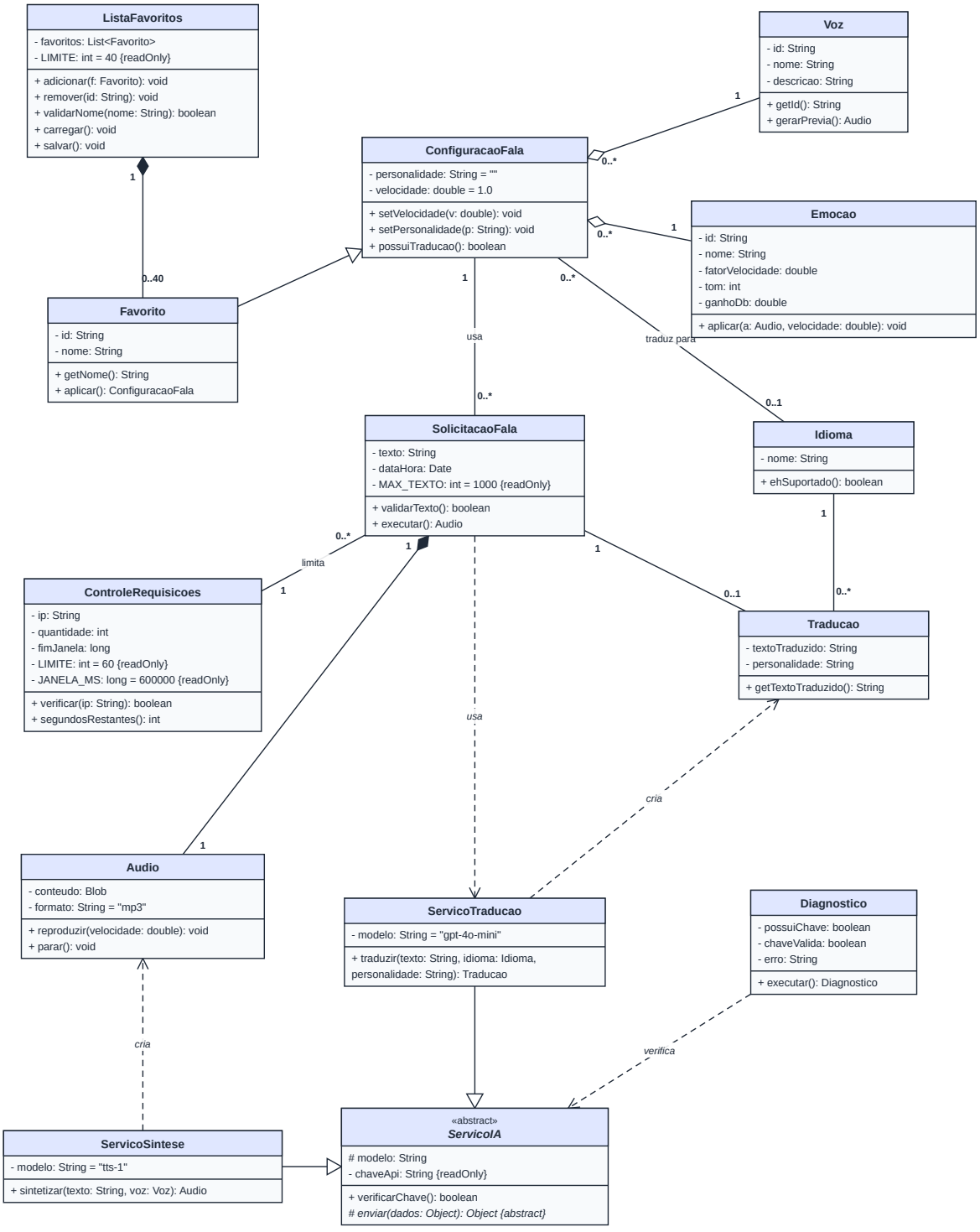
5. Diagrama de Classes de Projeto

Refinamento das classes de análise (seção 4) com a notação completa da UML: visibilidade, tipos, valores iniciais, propriedades e os tipos de relacionamento.

5.1 Da análise ao projeto

Classe de análise	Classe de projeto	O que mudou
`Voz` (identificador, nome, descricao)	`Voz` (-id: String, -nome: String, -descricao: String, +gerarPrevia(): Audio)	Tipos, visibilidade e retorno dos métodos
`ConfiguracaoFala` (idioma, personalidade, velocidade)	`ConfiguracaoFala` (-velocidade: double = 1.0, ...)	O atributo `idioma` virou associação com a nova classe `Idioma`; valores iniciais definidos
`Favorito` (nome, dataCriacao) + associação 1—1 com `ConfiguracaoFala`	`Favorito` **é uma** `ConfiguracaoFala`	A associação 1—1 virou **generalização**: um favorito é uma configuração com nome
`Traducao` (textoOriginal, textoTraduzido, idiomaDestino)	`Traducao` como **classe associativa**	Existe na relação entre `SolicitacaoFala` e `Idioma`
Ator externo OpenAI	`ServicoIA` «abstract», `ServicoTraducao`, `ServicoSintese`	Classes de fronteira que encapsulam o serviço externo

5.2 Diagrama



Arquivo editável no Draw.io: `diagramas/classes-projeto.drawio`

5.3 Classes

ConfiguracaoFala

Membro	Descrição
`- personalidade: String = ""`	Personalidade da tradução, até 100 caracteres (RN-06)
`- velocidade: double = 1.0`	Velocidade de reprodução entre 0,6 e 1,6 (RN-07)
`+ setVelocidade(v: double): void`	Altera a velocidade respeitando a faixa permitida
`+ setPersonalidade(p: String): void`	Altera a personalidade respeitando o tamanho máximo
`+ possuiTraducao(): boolean`	Retorna verdadeiro quando há idioma de destino associado

Favorito *(subclasse de ConfiguracaoFala)*

Membro	Descrição
`- id: String`	Identificador único (UUID)
`- nome: String`	Nome único e obrigatório (RN-09)
`+ aplicar(): ConfiguracaoFala`	Devolve a configuração para preencher a tela

ListaFavoritos

Membro	Descrição
`- favoritos: List<Favorito>`	Favoritos do navegador, do mais novo para o mais antigo
`- LIMITE: int = 40 {readOnly}`	Quantidade máxima (RN-08)
`+ adicionar(f: Favorito): void`	Insere no topo e descarta o excedente
`+ remover(id: String): void`	Exclui um favorito
`+ validarNome(nome: String): boolean`	Verifica se o nome não é vazio nem repetido
`+ carregar(): void` / `+ salvar(): void`	Lê e grava a lista no `localStorage`

Voz, Emocao e Idioma

Classe	Membros principais
`Voz`	`- id`, `- nome`, `- descricao: String`; `+ gerarPrevia(): Audio`
`Emocao`	`- fatorVelocidade: double`, `- tom: int` (cents), `- ganhoDb: double`; `+ aplicar(a: Audio, velocidade: double): void`
`Idioma`	`- nome: String`; `+ ehSuportado(): boolean` (RN-03)

SolicitacaoFala, Traducao e Audio

Classe	Membros principais
`SolicitacaoFala`	`- texto: String`, `- dataHora: Date`, `- MAX_TEXTO: int = 1000 {readOnly}`; `+ validarTexto(): boolean`, `+ executar(): Audio`
`Traducao`	`- textoTraduzido: String`, `- personalidade: String`; `+ getTextoTraduzido(): String`
`Audio`	`- conteudo: Blob`, `- formato: String = "mp3"`; `+ reproduzir(velocidade: double): void`, `+ parar(): void`

Serviços e controle

Classe	Membros principais
`ServicoIA` «abstract»	`# modelo: String`, `- chaveApi: String {readOnly}`; `+ verificarChave(): boolean`, `# enviar(dados: Object): Object {abstract}`
`ServicoTraducao`	`- modelo = "gpt-4o-mini"`; `+ traduzir(texto: String, idioma: Idioma, personalidade: String): Traducao`
`ServicoSintese`	`- modelo = "tts-1"`; `+ sintetizar(texto: String, voz: Voz): Audio`
`ControleRequisicoes`	`- ip: String`, `- quantidade: int`, `- fimJanela: long`, `- LIMITE: int = 60 {readOnly}`, `- JANELA_MS: long = 600000 {readOnly}`; `+ verificar(ip: String): boolean`, `+ segundosRestantes(): int`
`Diagnostico`	`- possuiChave: boolean`, `- chaveValida: boolean`, `- erro: String`; `+ executar(): Diagnostico`

5.4 Relacionamentos (aplicando a árvore de decisão)

Relacionamento	Tipo	Justificativa
`Favorito` → `ConfiguracaoFala`	**Generalização**	Um favorito **é uma** configuração de fala com nome e id
`ServicoTraducao` / `ServicoSintese` → `ServicoIA`	**Generalização**	Ambos **são** serviços de IA; cada um implementa `enviar()` com um modelo diferente. `ServicoIA` é abstrata e nunca é instanciada
`ListaFavoritos` ◆— `Favorito`	**Composição** (`1` — `0..40`)	A parte não sobrevive sem o todo: ao limpar a lista do navegador, os favoritos deixam de existir
`SolicitacaoFala` ◆— `Audio`	**Composição** (`1` — `1`)	O áudio só existe como resultado de uma solicitação; descartada a solicitação, o áudio é descartado

Relacionamento	Tipo	Justificativa
`ConfiguracaoFala` ◇— `Voz`	**Agregação** (`0..*` — `1`)	A configuração "tem uma" voz, mas a voz sobrevive sem a configuração e é compartilhada por várias
`ConfiguracaoFala` ◇— `Emocao`	**Agregação** (`0..*` — `1`)	Mesmo raciocínio da voz
`ConfiguracaoFala` — `Idioma`	**Associação** (`0..*` — `0..1`)	"Traduz para": zero idiomas quando "Sem tradução"
`SolicitacaoFala` — `ConfiguracaoFala`	**Associação** (`0..*` — `1`)	A solicitação usa a configuração atual, sem ser parte dela
`ControleRequisicoes` — `SolicitacaoFala`	**Associação** (`1` — `0..*`)	O controle de um IP limita as solicitações feitas por ele
`Traducao` entre `SolicitacaoFala` e `Idioma`	**Classe associativa**	`textoTraduzido` e `personalidade` não pertencem nem só à solicitação nem só ao idioma: existem na relação entre os dois
`SolicitacaoFala` ⇢ `ServicoTraducao`	**Dependência**	Vínculo temporário: o serviço é usado durante `executar()`
`ServicoTraducao` ⇢ `Traducao`, `ServicoSintese` ⇢ `Audio`	**Dependência**	Os serviços criam esses objetos como retorno dos métodos
`Diagnostico` ⇢ `ServicoIA`	**Dependência**	Usa `verificarChave()` durante `executar()`

5.5 Checklist de refinamento

Item	Verificação
Dinâmico	Todo caso de uso tem classes que o suportam: UC01 → `SolicitacaoFala` / `ServicoSintese` / `Audio`; UC02/UC03 → `ServicoTraducao` / `Traducao`; UC04 → `Emocao`; UC05/UC06 → `Voz`; UC07 → `ConfiguracaoFala`; UC08–UC10 → `ListaFavoritos` / `Favorito`; UC11 → `Diagnostico`; UC13 → `ControleRequisicoes`
Estático	Todos os atributos têm tipo e visibilidade; constantes marcadas com `{readOnly}`
Estrutural	Toda associação tem multiplicidade nas duas pontas; não há classes soltas sem relação
Nomenclatura	Classes no singular em PascalCase; atributos e métodos em camelCase; métodos com verbo
Duplicidade	"Preset" e "Favorito" unificados; `idioma` deixou de ser atributo repetido em duas classes e virou a classe `Idioma`

5.6 Correspondência com o código

Classe	Implementação
`Voz`, `Emocao`, `Idioma`	`lib/constants.ts` (`VOICES`, `EMOTIONS`, `LANGUAGES`)
`Emocao.aplicar()`, `Audio.reproduzir()` / `parar()`	`lib/audio.ts` (`playBlobWithEmotion`, `stopEmotionPlayback`)
`Favorito`, `ListaFavoritos`	`lib/presets.ts`
`ControleRequisicoes`	`lib/ratelimit.ts`
`ServicoTraducao`	`app/api/translate/route.ts`
`ServicoSintese`	`app/api/tts/route.ts`
`Diagnostico`	`app/api/diag/route.ts`
`SolicitacaoFala`, `ConfiguracaoFala`	Estado e função `handleSpeak` em `app/page.tsx`

6. Versionamento com Git

O projeto é versionado com Git e hospedado no GitHub (`TheObscuro/Tradutor-Neox`), atendendo ao RNF-10.

6.1 Configuração

```
git config --global user.name "VitorManoel"
git config --global user.email "seuemail@exemplo.com"
git config --list
```

O `.gitignore` impede que arquivos gerados ou sensíveis entrem no histórico:

Padrão	Motivo
`node_modules/`	Dependências reconstruídas com `npm install`
`.next/`, `out/`, `dist/`	Saída de build
`.env`, `.env.local`, `.env.*.local`	Contêm a chave da API (RNF-07)
`*.mp3`, `*.wav`, `*.zip`	Áudios e arquivos binários pesados

O `package-lock.json` **é versionado**, para que todos instalem as mesmas versões das dependências.

6.2 Commits semânticos (Conventional Commits)

Formato adotado: `<tipo>(escopo opcional): <descricao curta>`, verbo no presente, letras minúsculas, sem ponto final e sem acentos. Quando o motivo da mudança não é óbvio, o commit tem corpo (segundo `-m`).

Tipo	Exemplo real do projeto
`feat`	`feat(tts): valida voz e tamanho do texto e aplica limite de requisicoes`
`feat`	`feat(ui): unifica tela com favoritos, velocidade, previa de voz e personalidade`
`fix`	`fix(api): renomeia rota de diagnóstico para route.ts`
`docs`	`docs: adiciona requisitos funcionais, nao funcionais e regras de negocio`
`style`	`style: adiciona folha de estilos global com layout responsivo`
`refactor`	`refactor(audio): extrai reproducao com emocao para lib/audio`
`chore`	`chore(scripts): adiciona gerador de diagramas UML em drawio e svg`

Exemplo de commit com corpo:

```
git commit -m "refactor: centraliza vozes, idiomas e limites em lib/constants" \
  -m "A rota de vozes passa a ler a lista de lib/constants.ts, que tambem sera usada
pelo front-end e pelas validacoes das rotas."
```

> Os primeiros commits do repositório (`first commit`, `adiciona dependências do TypeScript`) são anteriores à adoção do padrão. Como já foram enviados ao GitHub, não foram reescritos — reescrever histórico compartilhado causa problemas para quem já tem o repositório clonado.

6.3 Branches

Cada tarefa é desenvolvida em uma branch própria, com o tipo do commit como prefixo, e integrada com `merge`:

Branch	Conteúdo
`docs/resumos-aulas`	Resumos das aulas 2 a 9
`feat/validacoes-api`	Constantes compartilhadas, validações, limite de requisições, personalidade, áudio e favoritos
`feat/tela-unificada`	Estilos globais e tela única com todos os recursos
`docs/requisitos`	Requisitos funcionais, não funcionais e regras de negócio
`feat/aviso-voz-ia`	Aviso de voz gerada por IA (RNF-11)
`docs/casos-de-uso`	Gerador de diagramas, diagrama e documentação dos casos de uso
`docs/diagrama-de-classes`	Classes de análise e de projeto
`docs/versionamento-e-pdf`	Esta seção e a geração do PDF

Fluxo usado em cada tarefa

```
git status                              # 1. repositorio limpo
git switch -c docs/requisitos           # 2. cria a branch da tarefa
git add docs/02-requisitos.md           # 3. commits pequenos e semanticos
git commit -m "docs: adiciona requisitos funcionais, nao funcionais e regras de negocio"
git switch claude/inspiring-faraday-wec0ca   # 4. volta para a branch de integracao
git merge --no-ff docs/requisitos       #    e integra a tarefa
git branch -d docs/requisitos           # 5. exclui a branch ja integrada
git push origin claude/inspiring-faraday-wec0ca
```

O `--no-ff` força a criação do commit de merge mesmo quando seria possível um *fast-forward*, para que o grafo mostre claramente onde cada tarefa começou e terminou.

A branch de integração é publicada no GitHub e entra na `main` por **pull request** (TheObscuro/Tradutor-Neox#1), o que permite revisar o código antes de integrá-lo e gera automaticamente uma prévia do site na Vercel.

Grafo do histórico

Trecho de `git log --oneline --graph`:

```
*   Merge branch 'docs/diagrama-de-classes'
|\
| * docs: documenta classes de projeto e seus relacionamentos
| * docs(uml): adiciona diagrama de classes de projeto
| * docs: documenta substantivos candidatos e classes de analise
| * docs(uml): adiciona diagrama de classes de analise
|/
*   Merge branch 'docs/casos-de-uso'
|\
| * docs: documenta atores, casos de uso e seus fluxos
| * docs(uml): adiciona diagrama de casos de uso
| * chore(scripts): adiciona gerador de diagramas UML em drawio e svg
|/
*   Merge branch 'feat/aviso-voz-ia'
|\
| * feat(ui): informa que a voz reproduzida e gerada por IA
|/
*   Merge branch 'docs/requisitos'
|\
| * docs: adiciona requisitos funcionais, nao funcionais e regras de negocio
|/
*   Merge branch 'feat/tela-unificada'
|\
| * refactor: remove tela duplicada da raiz do projeto
| * feat(ui): unifica tela com favoritos, velocidade, previa de voz e personalidade
| * style: adiciona folha de estilos global com layout responsivo
|/
*   Merge branch 'feat/validacoes-api'
|\
| * feat(presets): adiciona funcoes de favoritos salvos no navegador
| * refactor(audio): extrai reproducao com emocao para lib/audio
| * feat(tts): valida voz e tamanho do texto e aplica limite de requisicoes
| * feat(translate): valida entrada, aplica limite de requisicoes e personalidade
| * refactor: centraliza vozes, idiomas e limites em lib/constants
|/
*   Merge branch 'docs/resumos-aulas'
```